

Toward a Super-MeTTa Runtime: A Conceptual Primer on Layered Search, Typed Elaboration, Indexed Spaces, Oracle Boundaries, Attention-Aware Memory, and Benchmarking

2026-03-30

Abstract

This note distills a coherent architecture for a next-generation MeTTa runtime aimed at symbolic reasoning, large indexed knowledge spaces, future dependent-type elaboration, clean foreign-oracle boundaries, and eventually attention-aware memory management. The main thesis is that a durable design comes from separating *hard semantics* from *soft policy*. Hard semantics includes ownership, lifetimes, profiles, binding identity, and the boundaries between local search, shared indexed spaces, and elaboration. Soft policy includes indexing choices, scheduling, caching, search strategy, and attention allocation. The result is a conceptual blueprint for a *Super-MeTTa* runtime: deterministic by default, nondeterministic when asked, index-friendly at large scale, profile-driven rather than fork-driven, and prepared for later memory economics and external reasoning oracles without baking those concerns into the semantic core.

1 Purpose and Scope

The goal of this primer is not to document one particular commit or implementation snapshot. It is to record the interesting conceptual ideas needed to guide later work and later conversations. It therefore focuses on:

- the architecture of a serious symbolic runtime rather than one benchmark hack,
- the separation of machines and policies,
- the relationship between search, types, spaces, parsing, oracles, memory, and concurrency,
- and the order in which these pieces should be built so that local improvements do not sabotage the long-term system.

The intended setting is a MeTTa family runtime, but the principles are broader. They apply to any symbolic system that wants to support: (1) local backtracking-style reasoning, (2) large indexed shared spaces, (3) profile-specific language surfaces, (4) optional theorem proving and solver integration, and (5) long-lived interactive or agentic workloads.

2 The Central Design Thesis

The runtime should be organized around the following distinction:

Hard semantics should answer what a value *is*, who owns it, where it can live, and what guarantees a computation exposes.

Soft policy should answer how the runtime prioritizes, schedules, indexes, caches, or evicts within those semantic guarantees.

This one distinction clarifies many otherwise confusing debates:

- **Determinism** is semantic; clause indexing is policy.
- **A promoted session object** is semantic; its eviction priority is policy.
- **A profile's available forms and effects** are semantic; the search strategy used to execute them can be policy.
- **A source/sink/program-context boundary** is semantic; the batching and scheduling policy across that boundary is policy.

A useful slogan is:

Make user-visible meaning explicit; keep backend cleverness mostly invisible.

3 A Three-Machine Runtime with a Profile Layer

A robust symbolic runtime should not be one giant mutable evaluator. It should be factored into three distinct machines plus a profile layer.

3.1 Machine 1: the local search machine

This machine handles rollback-heavy local symbolic computation:

- unification and matching,
- branch-local bindings,
- nondeterministic choice points,
- backtracking and rollback,
- and branch-local scratch allocation.

Its implementation can learn from the WAM without becoming a full PROLOG clone. The important lessons are:

- use trails or equivalent rollback structures,
- tie scratch allocation to choice-frame watermarks,
- avoid repeated full cloning of environments,
- and use indexing to reduce pointless branching, not to define semantics.

3.2 Machine 2: the indexed shared-space machine

This machine handles persistent, queryable, largely shared state:

- exact and prefix lookup,
- indexed joins and conjunctions,
- snapshots and epochs,
- result batches,
- and external program/source/sink execution over shared data.

This is where MORK, path maps, zippers, or other large-scale indexed substrates belong. The point of this machine is not to mimic the local evaluator. It is to make large read-mostly spaces cheap to share, cheap to query, and amenable to later parallelism.

3.3 Machine 3: the type/elaboration machine

This machine handles:

- scoped binders,
- metavariables,
- type constraints,
- normalization and definitional equality,
- and elaboration from surface forms to a kernel.

It may share syntax and identifiers with the runtime, but it should not reuse the runtime’s local search environment as its main state representation. It is another rollbackable machine, but with different invariants and different error conditions.

3.4 Profiles above the machines

A *profile* is a packaged language surface assembled over the machines. A profile should specify:

- surface syntax,
- callable/effect inventory,
- evaluation policy,
- type policy,
- standard library,
- capability set,
- and optional oracle/resource access.

Examples include a base METTA profile, an extended HE-PRIME profile, a future PETTA profile, and future PROLOG-inspired profiles.

Design consequence. Language variants should be profiles, not evaluator forks.

4 Identifiers: Global Symbols versus Binder Identity

A runtime of this kind needs two distinct identity spaces.

4.1 Global spelling identity: `SymbolId`

All persistent symbol spellings should be interned and represented by compact symbol ids. Strings should remain boundary phenomena: parser input, printer output, serialization, and diagnostics.

The consequences are significant:

- symbol equality becomes integer equality,
- dispatch can be descriptor-driven rather than string-driven,
- indexing becomes denser and cheaper,
- streaming parsers can intern byte spans directly,
- and metadata tables can be keyed by symbol id.

4.2 Binder identity: `VarId`

Variables are not just symbols. A variable needs a stable binder identity, often with a surface spelling only for display or debugging. In other words:

- `SymbolId` answers *what global spelling is this?*
- `VarId` answers *which binder is this occurrence attached to?*

That separation is foundational. It prevents the runtime from conflating user spelling with semantic scope.

5 Namespaces and Module Qualification

A library ecosystem benefits from qualified names. The cleanest long-term solution is:

Use one canonical semantic namespace separator, and treat prettier variants only as surface sugar.

A practical choice is to keep names internally canonicalized as

`math:exp` `mm2:sink` `mork:context-run!`

and optionally allow the reader or printer to accept or display

`math.exp` `mm2.sink`

provided the dotted form is immediately canonicalized to the colon form before symbol interning. The benefit is that there remains only one semantic algebra for:

- module paths,

- exported names,
- inventories,
- and compatibility layers.

The runtime should *not* develop two independent qualification systems.

6 Deterministic by Default, Search by Opt-In

A core design goal is to avoid the classic backend-tuning burden often associated with PROLOG-style systems. The runtime should therefore distinguish two user-facing modes.

6.1 Default evaluator

The default evaluator should be deterministic by default, or at least explicit about nondeterminism. Users should be able to control:

- whether they want the first answer, all answers, or an aggregate,
- whether result order is stable, fair, or unspecified,
- and whether memoization/tabling is off, on, or automatic.

What users should *not* need to control routinely:

- trie shape,
- join order quirks,
- exact clause indexing policy,
- or backend-specific term layout tricks.

6.2 Separate search service

At the same time, the runtime should admit a more ATP-like search engine or search service for cases where aggressive search is appropriate:

- proof search,
- auto tactics,
- large branching symbolic synthesis,
- weighted or portfolio search,
- and result prioritization.

The main principle is simple:

Ordinary programming should not feel like backend tuning. Aggressive search should be opt-in.

7 Shared Spaces: Space Discipline versus Space Engine

An important architectural refinement is to separate: (1) the *discipline* of a space from (2) the *engine* used to implement it.

For example:

- a discipline might be `atom`, `stack`, `queue`, or `hash`,
- while an engine might be `native`, `indexed`, or `mork-like`.

This matters because an indexed engine is not semantically “another stack kind.” It is an implementation substrate for storage and query.

The right public abstraction is therefore something like:

- **discipline**: what user-level store behavior is promised,
- **engine**: how the store is represented and queried,
- **capabilities**: what operations the store supports.

8 Programs, Sources, Sinks, and External Oracles

External integrations should be organized around semantically honest nouns.

8.1 Programs

A program is a structured executable artifact: a rule set, a dataflow, or a declarative process specification.

8.2 Sources and sinks

Sources and sinks are *descriptors of interaction*, not necessarily open handles. They describe:

- where data comes from,
- where data should be sent,
- or what shape an external interaction has.

8.3 Resources and handles

Only things with real lifecycle or ownership should become handle-like resources:

- files,
- memory-mapped datasets,
- solver sessions,
- network endpoints,
- or external process contexts.

8.4 Metamath, SMT, ATPs

A theorem system such as Metamath should therefore normally appear as:

- a streamed source,
- a sink or checker,
- or an oracle/session,

not as a native kernel that the runtime pretends is just another library.

General rule. Do not name something `resource` or `open` unless it really owns or acquires something.

9 Parsing and Interchange: Stream First, Materialize Late

For large inputs and long-running agents, full-file materialization is the wrong default. The correct base pattern is:

stream input at coarse semantic granularity, and only promote what needs to survive.

This suggests a parser architecture with:

- buffered or memory-mapped text sources,
- slice-based tokenization,
- direct span-to-SymbolId interning,
- statement iterators rather than full AST lists,
- and delayed promotion into persistent memory.

For domain languages such as Metamath or PROLOG, the runtime should prefer:

streaming frontend → normalized IR → lowering

over “read the whole file into generic runtime terms and hope for the best.”

10 Profiles for Alternate Language Surfaces

A powerful runtime should be able to host alternate source languages without becoming a language zoo. The correct place for alternate surfaces is the profile layer.

10.1 A future PeTTa profile

A future PETTA profile should be treated first as:

- a manifest,
- an oracle/conformance lane,
- and a shadow profile,

before it becomes a full runtime implementation. This keeps PETTA as a semantic anchor without forcing premature evaluator entanglement.

10.2 A future Prolog profile

Likewise, it is reasonable to host PROLOG as a frontend over the search machine:

$$\text{PROLOG source} \rightarrow \text{CoreProlog IR} \rightarrow \text{search-machine lowering.}$$

The lesson is that “the opposite of implementing PETTA on PROLOG” is not a naive source-to-source transpiler. It is a real frontend plus a normalized intermediate representation.

11 Memory: Lifetimes First, Collection Later

A language runtime that wants to become long-lived, interactive, and agentic needs a durable memory model. The strongest conceptual move is:

finish lifetime and ownership discipline first; only then decide how much garbage collection is still necessary.

11.1 Hard lifetime strata

A clean symbolic runtime benefits from at least four lifetime strata:

1. **runtime-global**: symbol tables, builtin descriptors, immutable global metadata,
2. **module-frozen**: loaded libraries, frozen rule bundles, compiled assets,
3. **session-owned**: spaces, caches, state cells, active resources,
4. **scratch or branch-local**: temporary terms, substitution products, short-lived results.

These strata are not merely performance tricks. They are ownership classes.

11.2 Promotion

Promotion between strata must be explicit. A scratch object should not survive accidentally; it should survive only by promotion into a longer-lived stratum.

11.3 Root-walk first

Before a collector is introduced, the runtime should define an explicit root-walk interface for session-owned structures:

- registry,
- spaces,
- module store,
- caches,
- resources,
- and active program contexts.

11.4 If a collector is later needed

If region discipline, promotion, and streaming are still not enough, the right next collector is generally a:

precise, non-moving, session-heap collector

rather than a process-wide, conservative, moving collector. That preserves pointer stability and keeps scratch allocation fast and simple.

12 Attention-Aware Memory as a Policy Layer

An AGI-oriented runtime may want attention-like resource allocation. The right way to integrate this is:

hard lifetimes underneath, soft attention economics above.

12.1 What should carry attention?

Not every raw atom. Instead, attention should attach to *persistent carriers* such as:

- space entries,
- cached results,
- query templates,
- indexed chunks,
- resources,
- and possibly promoted derived facts.

12.2 A useful mapping

Attention-like concepts can then be interpreted as:

- **hotness** or short-term importance: scheduling priority and residency in hot caches,
- **retention** or long-term importance: reluctance to evict or demote,
- **pinning**: never evict unless explicitly told to do so.

12.3 Hierarchical attention, not per-atom micro-management

A scalable system should attach attention at multiple levels:

- coarse budgets for spaces, modules, and caches,
- mid-scale budgets for query families or indexed chunk groups,
- and only then finer-grained per-entry attention inside an already active region.

This avoids the wasteful design of micro-managing every transient term.

13 Parallelism: Early Win, Correct Foundation

The best early parallelism win is not shared mutable search state. It is:

snapshot-parallel evaluation over immutable indexed spaces

The runtime should therefore be designed so that:

- workers read the same immutable space snapshot,
- each worker owns its own scratch arena, bindings builder, and local result buffer,
- and merge occurs only at explicit barriers.

This validates the ownership model early without forcing every local operation across a message boundary.

Important anti-pattern. Do not turn local symbolic operations into communication-style overhead merely because a distributed or process-calculus model is conceptually attractive. Keep local search local and shared spaces shared.

14 Benchmarking: Measure the Right Things Separately

A serious runtime needs benchmark families, not one benchmark mythology. The following lanes should be treated as distinct.

14.1 Ingest and indexing

Measure:

- atoms or facts loaded per second,
- index-build time,
- peak memory during load,
- and the difference between source formats (e.g. text vs indexed archive).

14.2 Indexed query

Measure:

- point queries,
- prefix queries,
- conjunctions,
- and join-heavy workloads over large spaces.

14.3 Local search

Use workloads such as symbolic puzzles, finite graph search, and theorem-style branchy derivation to test rollback discipline, substitution churn, and branch scheduling.

14.4 Streaming aggregation

Test workloads where many proof or evidence items must be combined under thresholds, confidence revision, or grouping. These should test streaming reduction, not merely list materialization.

14.5 Update and recompute

Test add/remove or changing evidence, then rerun queries. This measures snapshots, epochs, and cache invalidation.

14.6 Oracle and source/sink integration

Test streamed imports, external solver calls, theorem checkers, and large archival data sources as first-class benchmark families.

14.7 Parallel snapshot workloads

Later, add multi-seed or multi-branch workloads over one frozen large snapshot to test the correctness of the ownership model under parallelism.

Interpretive rule. If one benchmark is dominated by `collect` or result materialization, it is primarily testing an aggregation strategy, not a whole runtime philosophy.

15 A Clean Roadmap

A practical order of work follows from the architecture.

1. **Freeze the nouns and seams.** Finalize the split between local search, shared spaces, type/elaboration, profiles, and program/source/sink/resource concepts.
2. **Finish the local search machine.** Trail-like rollback, choice frames, scratch watermarks, and environment builders come before more benchmark chasing.
3. **Stabilize the space engine boundary.** Make the indexed shared-space substrate explicit and honest.
4. **Keep the foreign ABI layered and packet-oriented.**
Stable handle families and result batches should be favored over leaking internal data structures.
5. **Introduce the profile system.**
Alternate language surfaces and capability bundles should become profiles, not evaluator forks.
6. **Add shadow profiles and frontend lanes.**
Use future PETTA and PROLOG support first as conformance and frontend anchors before expanding runtime commitments.
7. **Finish the memory strata and promotion model.**
Then, only if still needed, add a precise non-moving session collector.
8. **Add the parallel snapshot MVP.**
This validates ownership and prepares for larger scaling work.

9. **Build the richer type/elaboration machine.**

This should be done as its own rollbackable machine rather than an extension of runtime bindings.

16 Anti-Patterns to Avoid

The following design mistakes are especially tempting and especially costly.

1. **One giant mutable evaluator.**

This entangles search, spaces, types, oracles, and policy.

2. **Two semantic namespace systems.**

One canonical qualifier is enough; prettier forms should be only sugar.

3. **Making every local operation a communication boundary.**

Keep shared-space and oracle boundaries coarse and semantically meaningful.

4. **Letting runtime strings remain the core identity model.**

Strings belong at boundaries; ids belong in the hot path.

5. **Using attention to replace ownership.**

Lifetimes and roots should be explicit whether or not attention exists.

6. **Treating one benchmark pathology as the whole architecture.**

Separate ingest, query, search, aggregation, and oracle lanes.

7. **Prematurely implementing a full alternate runtime profile.**

Shadow profiles and frontend lanes should come before a full semantic duplication.

17 Conclusion

A serious METTA runtime should not be thought of as one evaluator getting gradually more clever. It should be understood as a layered symbolic operating environment with:

- a local search machine for rollback-heavy symbolic work,
- an indexed shared-space machine for large persistent knowledge,
- a separate elaboration machine for future typed kernels,
- a profile layer for alternate language surfaces,
- explicit program/source/sink/resource seams for foreign integration,
- a lifetime-first memory discipline,
- and a policy layer for indexing, caching, scheduling, and later attention economics.

The simplest summary is:

Separate the machines, make ownership explicit, keep user-visible meaning stable, and let backend intelligence accumulate behind those boundaries.

That is the cleanest route to a *Super-MeTTa* runtime capable of supporting symbolic programming, large knowledge spaces, theorem interaction, solver integration, future type elaboration, and eventually more AGI-like memory economics without turning into an entangled benchmark-driven mess.

Selected References for Further Work

References

- [1] Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. MIT Press, 1991.
- [2] Mads Tofte and Jean-Pierre Talpin. Region-based memory management. *Information and Computation*, 132(2):109–176, 1997.
- [3] Frank McSherry, Derek Murray, Rebecca Isaacs, and Michael Isard. Differential dataflow. In *CIDR*, 2013.
- [4] Niko Matsakis and collaborators. *Datafrog: a lightweight Datalog engine*. Software/library documentation and implementation notes.
- [5] Jon Raffkind and John Regehr. Precise garbage collection for C. In *ISMM Workshop on Memory Systems Performance and Correctness*, 2009.
- [6] Kyren and contributors. *gc-arena: isolated garbage-collected arenas for runtime implementations*. Library documentation and design notes.
- [7] Jan Wielemaker and contributors. Just-in-time indexing and tabling facilities in SWI-Prolog. System documentation and implementation notes.
- [8] OpenCog documentation on economic attention allocation and later critiques of per-atom attention. Historical design notes.